CSE 565 Structural-Based Testing Project

Chase Coleman
Arizona State University
CSE 565: Software Verification and Validation

August 29, 2026

1 Introduction

This report presents the results of a structural-based testing assignment. Part 1 uses test cases for the provided `VendingMachine.java` program and discusses statement and decision coverage. Part 2 uses PMD to perform static source code analysis on `StaticAnalysis.java` and evaluates the anomalies detected by the tool.

2 Part 1: Vending Machine Testing

2.1 Tool Used and Coverage Type

For Part 1, JUnit 5 and JaCoCo were used to test and measure coverage for `VendingMachine.java`. JUnit 5 provides the assertion framework used to execute each test case and compare the actual output from `dispenseItem` against the expected output (JUnit Team, n.d.). JaCoCo is a Java code coverage tool that instruments the compiled program during test execution and produces coverage reports after the tests finish (JaCoCo Team, n.d.).

JaCoCo reports several coverage measurements, including instruction coverage, line coverage, branch coverage, complexity coverage, method coverage, and class coverage (JaCoCo Team, n.d.). For this assignment, line coverage is used as the practical measurement for statement coverage because it shows whether the executable lines of the program were run. Branch coverage is used as the practical measurement for decision coverage because it shows whether the true and false outcomes of conditional decisions were exercised.

The tests and coverage report were run with Maven, a Java build tool that manages project compilation, testing, and documentation through a project object model file (Apache Software Foundation, n.d.), using the following command:

Listing 1: JUnit and JaCoCo Test Command

```
mvn clean verify
```

2.2 Test Cases Developed

The test cases were selected to cover object construction, the available products, successful purchases, exact purchases, and insufficient-money cases. Table 1 summarizes the cases.

Table 1: Vending Machine Test Cases

Test	Input	Item	Expected Result
Construct vending machine	N/A	N/A	A `VendingMachine` object is created successfully
Candy with extra money	30	candy	Item dispensed and change of 10 returned
Coke exact money	25	coke	Item dispensed.
Coffee exact money	45	coffee	Item dispensed.
Coffee, enough for candy or coke	30	coffee	Item not dispensed, missing 15 cents. Can purchase candy or coke.
Coffee, enough for candy only	22	coffee	Item not dispensed, missing 23 cents. Can purchase candy.
Coffee, not enough for anything	10	coffee	Item not dispensed, missing 35 cents. Cannot purchase item.

2.3 JUnit Test Code

The following JUnit test code was written to execute the test cases. Each test uses assertions to confirm that the actual output from `dispenseItem` matches the expected result.

Listing 2: Vending Machine JUnit Tests

```
import org.junit.jupiter.api.Test;


import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
import static org.junit.jupiter.api.Assertions.assertNotNull;


class VendingMachineTest
{
    @Test
    void vendingMachineCanBeConstructed()
    {
        assertNotNull(new VendingMachine());
    }


    @Test
    void candyWithExtraMoneyReturnsItemAndChange()
    {
        assertEquals(
                "Item dispensed and change of 10 returned",
                VendingMachine.dispenseItem(30, "candy")
        );
    }


    @Test
    void cokeWithExactMoneyReturnsItem()
    {
        assertEquals(
                "Item dispensed.",
                VendingMachine.dispenseItem(25, "coke")
        );
    }


    @Test
```

```
void coffeeWithExactMoneyReturnsItem()
{
    assertEquals(
            "Item dispensed.",
            VendingMachine.dispenseItem(45, "coffee")
    );
}

@Test
void coffeeWithThirtyCentsReportsCandyOrCokeAlternative()
{
    assertEquals(
            "Item not dispensed, missing 15 cents. Can purchase candy or coke
.",
            VendingMachine.dispenseItem(30, "coffee")
    );
}

@Test
void coffeeWithTwentyTwoCentsReportsCandyAlternative()
{
    assertEquals(
            "Item not dispensed, missing 23 cents. Can purchase candy.",
            VendingMachine.dispenseItem(22, "coffee")
    );
}

@Test
void coffeeWithTenCentsReportsNoAffordableItem()
```

```
    {
        assertEquals(
                "Item not dispensed, missing 35 cents. Cannot purchase item.",
                VendingMachine.dispenseItem(10, "coffee")
        );
    }
}
```

2.4 Test Execution Result

The Maven test execution showed that all seven JUnit tests passed with no failures, errors, or skipped tests:

Listing 3: JUnit Test Execution Summary

```
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
All coverage checks have been met.
BUILD SUCCESS
```

Figure 1 shows the JUnit test execution output. The final JUnit test suite contains seven tests, including one construction test so that JaCoCo records coverage for the implicit constructor line in `VendingMachine.java`.

Figure 1: JUnit test execution output for the vending machine

2.5 Coverage Discussion

The test suite was designed to exercise the main control-flow paths in `dispenseItem`. The construction test covers the class's implicit default constructor, which JaCoCo reports as an executable line. The product tests cover the product selection statements for candy, coke, and coffee. The successful-purchase tests cover the `input > cost` and `input == cost` branches. The remaining three tests cover the insufficient-money branch and the nested decisions for `input < 45`, `input < 25`, and `input < 20`.

Together, these tests execute the statements that assign product costs, compute change, assign successful return messages, assign insufficient-money messages, and return the final result. The insufficient-money tests were intentionally chosen at 30, 22, and 10 cents so that the nested decision logic is exercised across its different outcomes.

The JaCoCo report showed 0 missed lines and 24 covered lines, giving 100% line coverage. It also showed 1 missed branch and 15 covered branches, giving 93.75% branch coverage. This satisfies the assignment goals of 100% statement coverage and at least 90% decision coverage.

The only missed branch is the false outcome of `if(input < 45)` inside the insufficient-funds block. For the valid product inputs, this false branch is unreachable. The insufficient-funds block only executes when `input < cost`, and the highest valid product cost is coffee at 45 cents. Therefore, when the insufficient-funds block is reached for coffee, `input` must already be less than 45.

One implementation issue is that the vending machine code compares strings using `==`. These tests use Java string literals such as `"candy"`, `"coke"`, and `"coffee"`, so the current implementation behaves as expected because string literals are interned. In production code, `.equals()` would be safer because `==` compares object references rather than string contents.

2.6 Coverage Report

The JaCoCo CSV report generated by the test run is shown below. The report shows that `VendingMachine` achieved 100% line coverage and 93.75% branch coverage.

Listing 4: JaCoCo Coverage Summary

```
GROUP,PACKAGE,CLASS,INSTRUCTION_MISSED,INSTRUCTION_COVERED,BRANCH_MISSED,
    BRANCH_COVERED,LINE_MISSED,LINE_COVERED,COMPLEXITY_MISSED,COMPLEXITY_COVERED,
    METHOD_MISSED,METHOD_COVERED
structural-based-testing-project,default,VendingMachine,0,71,1,15,0,24,1,9,0,2
```

Figure 2 shows the code coverage output achieved by the vending machine test cases.

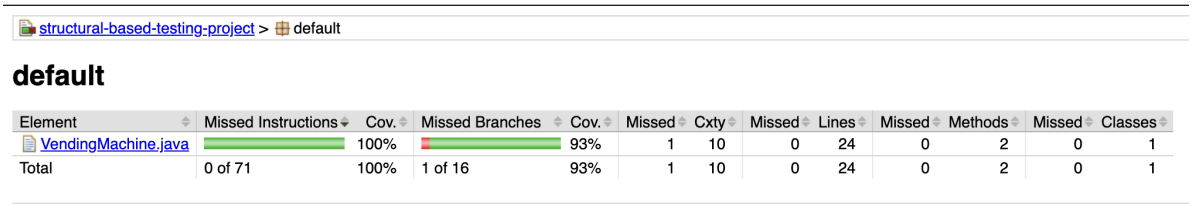

Figure 2: Vending machine code coverage result

3 Part 2: Static Analysis

3.1 Tool Used and Analysis Type

PMD 6.55.0 was used as the static source code analysis tool. PMD analyzes source code without executing the program. It can detect unused variables, questionable object comparisons, style issues, design issues, empty code blocks, duplicated code, overly complex methods, and other maintainability problems. For Java programs, PMD performs parsing, symbol resolution, and type resolution before rules inspect the source code (PMD Project, n.d.).

PMD was downloaded as a standalone distribution and run from the command line. The command used was:

Listing 5: PMD Command

```
/tmp/pmd-bin-6.55.0/bin/run.sh pmd -d StaticAnalysis.java -R rulesets/java/
    quickstart.xml -f text > pmd_static_analysis_report.txt
```

3.2 PMD Report

The PMD report generated for `StaticAnalysis.java` is shown below.

Listing 6: PMD Static Analysis Report

```
StaticAnalysis.java:4:  NoPackage:  All classes, interfaces, enums and
    annotations must belong to a named package
StaticAnalysis.java:5:  UseUtilityClass:    All methods are static.  Consider
    using a utility class instead. Alternatively, you could add a private
    constructor or make the class abstract to silence this warning.
StaticAnalysis.java:15: UnusedLocalVariable:    Avoid unused local variables such
     as 'weight'.
StaticAnalysis.java:16: UnusedLocalVariable:    Avoid unused local variables such
     as 'length'.
StaticAnalysis.java:24: CompareObjectsWithEquals:   Use equals() to compare
    object references.
StaticAnalysis.java:24: UseEqualsToCompareStrings:  Use equals() to compare
    strings instead of '==' or '!='
StaticAnalysis.java:34: ControlStatementBraces: This statement should have braces
StaticAnalysis.java:36: ControlStatementBraces: This statement should have braces
```

3.3 Anomalies Detected

PMD detected unused local variables in the constructor on lines 15 and 16:

```
int weight = 0;
```

```
String length = "";
```

These variables are assigned values but are never used. This is a data flow anomaly because the variables are defined without any later use.

PMD also detected a string comparison problem on line 24:

```
if(product == "Electronics")
```

This is risky because Java's `==` operator compares object references. String contents should normally be compared with `.equals()`, such as:

```
if ("Electronics".equals(product))
```

PMD also reported additional style and design findings, including no package declaration, a utility-class recommendation, and missing braces around control statements. These findings are useful, but the unused-variable and string-comparison findings are the most relevant anomalies for this assignment.

3.4 Assessment of PMD

PMD was effective for this assignment because it identified the major suspicious patterns in `StaticAnalysis.java`. The report included the line number, rule name, and a short description for each finding, which made the results easy to interpret.

The main features of PMD include command-line analysis, configurable rulesets, multiple output formats, and Java quality rules for correctness, maintainability, and style. PMD covers anomalies such as unused variables, questionable comparisons, empty statements, unnecessary code, and design problems.

The tool was easy to use because it required only a source file, a ruleset, and an output format. The main limitation is that PMD reports both important anomalies and minor style issues, so the user must interpret which findings are most relevant to the assignment.

4 Conclusion

The vending machine test cases exercise the main statement and decision paths in the provided program. The PMD static analysis identified the important anomalies in `StaticAnalysis.java`, especially unused variables and unsafe string comparison. Together, the two parts demonstrate both dynamic testing through executed test cases and static analysis through source code inspection.

References

Apache Software Foundation. (n.d.). *Welcome to Apache Maven*. Retrieved August 29, 2026, from https://maven.apache.org/

JaCoCo Team. (n.d.). *JaCoCo Maven plug-in*. Retrieved August 29, 2026, from https://www.jacoco.org/jacoco/trunk/doc/maven.html

JUnit Team. (n.d.). *JUnit 5 user guide*. Retrieved August 29, 2026, from https://junit.org/junit5/docs/5.12.0/user-guide/index.html

PMD Project. (n.d.). *Java support*. Retrieved August 29, 2026, from https://pmd.github.io/pmd/pmd_languages_java.html